

ZEPHYR: AN EFFICIENT AUDIO DENOISING SYSTEM USING SPIKING NEURAL NETWORKS ENABLED WITH A SPARSITY-AWARE FLEXIBLE FPGA PE ARRAY

Cheng-En Chang^{*‡}, Chi-Wei Kao^{*‡}, Chung-Lun Yang^{*‡}, Yan-Lin Jiang^{*‡}, Yi-Chen Huang^{*‡},
Sebastian Fieldhouse[†] and Kea-Tiong Tang^{*}

^{*} Department of Electrical Engineering, National Tsing Hua University, Taiwan

[†] College of Semiconductor Research, National Tsing Hua University, Taiwan

[‡] These authors contributed equally to this work.

ABSTRACT

In this work we look to neuromorphic computing to solve the power consumption problem that audio denoising neural networks face on edge devices like smartphones, wireless headphones and hearing aids. Spiking neural networks (SNNs) have the potential to solve this problem due to their high activation sparsity and low complexity, however many SOTA SNNs require hardware that supports a mixture of operations to be able to fully perform inference. To solve this problem, we convert SOTA audio denoising neural network Spiking-FullSubNet to a hardware friendly version showing that via QAT and activation function simplification we can achieve $\approx 28\times$ improvement in power consumption to 52.9nJ per 32ms audio frame when calculated for custom digital hardware in a 45nm process node. We then propose a digital circuit which by means of a sparsity-aware flexible PE array can perform inference of the heterogeneous compute load of Spiking-FullSubNet, and validate this circuit on a PYNQ-Z1 FPGA achieving a real-time factor of 0.727 at 100MHz.

Index Terms— SNN Accelerator, Audio Denoising, Neuromorphic Computing, Speech Enhancement

1. INTRODUCTION

Smartphones, headphones and hearing aids are all examples of highly power constrained edge devices that are used to run audio denoising artificial neural networks (ANNs) to improve user experience. However, these devices are severely power constrained relying on battery power with users expecting long battery life between recharges, whilst ANNs are computationally resource and power hungry. Motivated by this problem we look to neuromorphic computing for a solution. Spiking neural networks (SNNs) offer the *potential* benefit of lower power consumption, on the condition that they are paired with appropriate hardware. Thus, in this paper we take a hardware-software co-design approach to create a neuromorphic audio denoising system that we call Zephyr. To do so, we take SOTA audio denoising algorithm Spiking Full-SubNet (SFSN) [1], which we convert to a hardware friendly

version via QAT and activation function simplification and batch norm removal, followed by mapping it to a custom digital hardware design - this is an accelerator circuit which could theoretically be taped out in silicon but for this work we use an FPGA to perform validation and gather latency estimates. However, SFSN, like many SOTA SNN algorithms, whilst benefiting from high activation sparsity, requires a range of different operation types to perform inference [2] [3]. Spiking layers can often make use of pure addition, but encoding layers can require matrix multiplication, and leakage terms often require elementwise multiplication. As such, in this work we also present a **heterogeneous** processing-element (PE) array, paired with an event-based memory read mechanism, to flexibly support the operations required by SFSN whilst also taking advantage of its activation sparsity.

2. BACKGROUND AND RELATED WORK

Neural networks provide a data-driven way to improve on-device audio denoising algorithms that can be optimised against perceptual objectives. ANNs such as RNNoise [4], DCCRN [5] and FullSubNet [6] have shown that using a neural network in adjunction with traditional techniques can give an improvement in audio quality, however these ANNs require hardware support in the device upon which they are deployed. Specifically, large models require large memory resources and ANNs require a lot of dense MAC operations - this can make them awkward to deploy on highly power constrained SoCs like those found in wireless headphones. Recent algorithms such as SFSN [1] and DPSNN [2] seek to decrease computational complexity by using SNNs with sparse, binary activations. Whilst these have shown good theoretical power consumption and computational efficiency whilst maintaining high performance (15.20dB and 14.70dB on the Intel-DNS challenge dataset for SFSN and DPSNN respectively), SNNs require hardware that is able to support their operations whilst also being able to take advantage of their high activation sparsity and low computational complexity to be able to actualize these power consumption gains.

Table 1. Speech-enhancement performance and energy efficiency on the Intel N-DNS Challenge dataset [7]. Energy costs for ANN/SNN baselines follow the Intel N-DNS operation-based energy-proxy methodology.

Model	Year / Rank	Params (K) (precision)	SI-SNR (dB) ↑	DNSMOS OVR ↑	Energy Cost ↓
<i>Official Intel N-DNS baselines</i>					
Microsoft NsNet2 [8]	2023	2,681 (FP32)	11.63	2.95	12.51 μ J
Intel DNS Network [7]	2023	1,901 (FP32)	12.51	3.08	—
SDNN Network [7]	2023	525 (FP32)	12.26	2.70	0.41 μ J
<i>ANN baselines evaluated on Intel N-DNS</i>					
Fast FullSubNet [9]	2023	1,141 (FP32)	13.88	2.91	0.09 mJ
CMGAN [10]	2024	1,413 (FP32)	14.01	2.81	1.47 mJ
S5 (sparse-11) [11]	2025	500-4K(INT8)	15.20	—	0.463 μ J [†]
<i>SNN baselines evaluated on Intel N-DNS</i>					
DPSNN [2]	2024	1400 (FP32)	14.70	2.90	1.08 μ J
<i>Intel N-DNS Challenge top-ranking systems</i>					
CTDNN LAVADL	Rank 2	905 (FP32)	13.52	2.97	1.76 μ J
Sparsity SDNN	Rank 3	344 (FP32)	12.16	2.70	0.27 μ J
PSNN	Rank 4	724 (FP32)	12.32	2.68	1.65 μ J
Spiking-FullSubNet-XL	Winner	965 (FP32)	15.20	3.03	1.48 μ J
Zephyr	Proposed	965 (INT8)	14.54	3.00	52.9 nJ

[†]S5 sparse-11 achieves the same 15.20 dB SI-SNR as Spiking-FullSubNet-XL with $3.2\times$ lower effective compute and $5.37\times$ lower memory less memory at iso-accuracy [11]. Its energy cost is estimated as $1.48/3.2 = 0.463 \mu\text{J}$ using the Intel N-DNS operation-based energy-proxy methodology, assuming that energy scales proportionally with effective operation count. This value is derived from the reported operation count rather than obtained from a direct hardware measurement. The proxy is based on a 45-nm CMOS energy model of 4.6 pJ per FP32 MAC and 0.9 pJ per AC operation.

3. HARDWARE-FRIENDLY DENOISING ALGORITHM

3.1. Spiking-FullSubNet Architecture Overview

In this work, we build on SFSN, whose building-block element is the gated-spiking neuron (GSN), as described in Sec. 3.2. SFSN combines a full-band GSN encoder with three sub-band branches, as seen in Fig. 1.

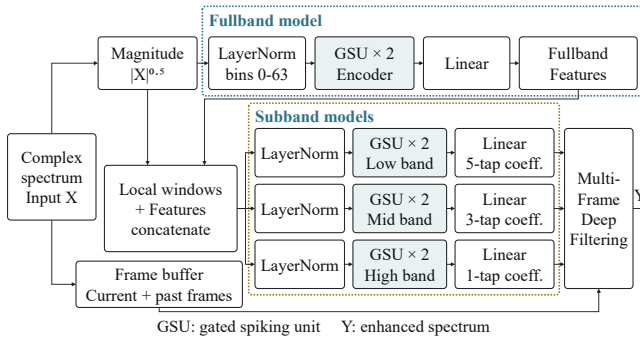


Fig. 1. Model data flow of Spiking-FullSubNet.

The full-band branch extracts a global spectral embedding, while each sub-band branch combines this embedding with neighboring frequency bins to capture local spectral structure. The low-, mid-, and high-band branches predict complex filtering coefficients with 5, 3, and 1 taps, respec-

tively. Multi-Frame Deep Filtering (MFDF) applies these coefficients to the current and preceding STFT frames to reconstruct the enhanced spectrum. This data flow requires dense matrix–vector products, spike-driven accumulation, gated state updates, batch and layer normalization, and complex multiply–accumulate operations.

3.2. GSN Membrane Update: Original Formulation

The original GSN neuron equation, before quantization, is given by

$$i^l(t) = W_{mn}^l o^{l-1}(t) + W_{nn}^l o^l(t-1) + b^l, \quad (1)$$

$$\lambda^l(t) = \sigma(W_{mn}^l o^{l-1}(t) + W_{nn}^l o^l(t-1) + \tilde{b}^l), \quad (2)$$

$$u^l(t) = \lambda^l(t) u^l(t-1) + (1 - \lambda^l(t)) i^l(t). \quad (3)$$

where $\lambda^l(t)$ is a dynamic decay gate, $u^l(t-1)$ is the previous membrane potential, $i^l(t)$ is the input current. The gate controls the relative contributions of the previous state and the current input. In the floating-point implementation, each membrane update involves two element-wise multiplications. Zephyr preserves this update structure while implementing the gating products and subsequent rescaling with integer arithmetic.

3.3. Hardware-Friendly Implementation Methods

For edge deployment, Zephyr applies QAT-based symmetric signed INT8 quantization to the neural-network weights, all

non-spiking activations, and recurrent membrane states. Matrix operations accumulate in INT32. Linear and GSN matrix weights use per-output-channel scales, whereas continuous activations use per-tensor scales and recurrent membrane states use per-channel scales. Binary spikes are represented separately from these continuous activations. The quantization step reduces weights from 4MB to 1MB, this is an important step as 4MB would require an unfeasible amount of SRAM for a low power edge SoC like those used in hearing aids or bluetooth headphones. For the GSN membrane update, the two gated products use wider integer intermediate precision. A 255-entry sigmoid LUT maps signed preactivations in $[-127, 127]$ to unsigned 8-bit gate coefficients with denominator 255, avoiding runtime exponential evaluation. Precomputed integer multipliers and shifts align the scales of the gated products and subsequent state updates. The batch-normalization affine transform following the membrane update is merged with the corresponding bias and requantization parameters to further decrease computation. This folding is distinct from the data-dependent LayerNorm operations in the full-band and sub-band paths.

3.4. Energy Analysis Methodology

For comparison with prior Intel N-DNS Challenge submissions, the operation-based power proxy is expressed as [7, 1]

$$P_{\text{proxy}} = \text{SynOps} + 10 \times \text{NeuronOps}, \quad (4)$$

where SynOps and NeuronOps denote synaptic and neuron operation rates, respectively. The weighting factor represents the relative cost assigned to neuron operations in the proxy. This metric captures changes in operation counts but does not explicitly account for operand precision, LUT access, or channel-wise requantization. Computational energy is estimated by weighting operation counts after zero-skipping by their corresponding costs. For 45 nm, 0.9 V CMOS, INT8 multiplication and INT32 addition are assigned 0.20 pJ and 0.10 pJ per operation, respectively [12]. Full-model accounting must also include membrane updates, channel-wise rescaling, LUT access, normalization, and deep filtering; these arithmetic coefficients alone do not capture memory-access costs. Since SFSN already accounts for spike activity [1], consistent counting rules are required to avoid double-counting sparsity savings. This cost-model estimate does not represent measured FPGA energy, rather the computation energy that could be achieved in custom silicon.

4. SPARSITY-AWARE FLEXIBLE HETEROGENOUS FPGA ACCELERATOR

4.1. Accelerator Architecture

As illustrated in Fig. 2, the proposed Zephyr accelerator comprises three parts: an inference module, a finite state machine and a event driven weight reading memory. A top-level

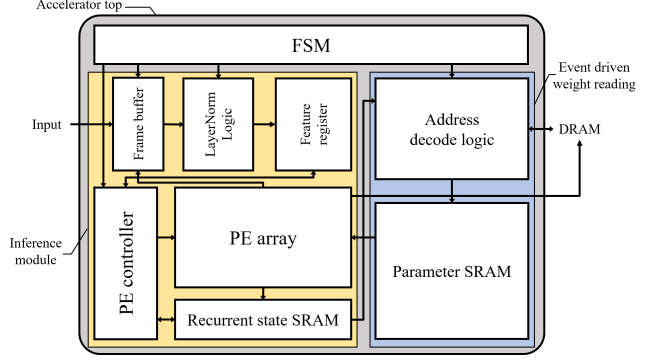


Fig. 2. Overall architecture of the Zephyr accelerator.

finite-state machine (FSM) coordinates their operation. The inference module executes the hardware-friendly audio denoising algorithm described in Section 3. It integrates frame buffering, layer normalization, feature register, a processing-element (PE) array and recurrent-state SRAM. The PE controller schedules computations on the PE array across the different inference stages. The event driven weight reading memory subsystem manages weight storage and access. It comprises a address decoding logic, and BRAM-based parameter memory. During matrix-spike computation, the address decoding logic translates stored active-neuron indices into the corresponding DRAM addresses for weight retrieval. The parameter memory buffers on-chip weight storage and supplies the PE array with the weights required for computation. By restricting weight retrieval and accumulation to active neuron events, this subsystem exploits activation sparsity to avoid unnecessary DRAM accesses and PE operations.

4.2. PE Array Architecture

Unlike conventional neural network accelerators that mainly target homogeneous multiply-accumulate (MAC) operations, this SNN, like many others from the literature, has a highly heterogeneous computation pattern. Thus, we design a flexible PE array capable of supporting multiple types of operations, including matrix multiplication, pure addition and element-wise multiplication (Multi-Frame Deep Filtering). As illustrated in Fig. 3, the same Hybrid PE array is reused via time-multiplexing across different computation phases. The three PE arrays shown in the figure represent different usages of the same physical hardware, rather than three independent PE arrays. During gate computation, the PE array performs the required MAC operations, after which the accumulated results are processed by the requantization bank and gate postprocessing unit. The gate value is then mapped through the sigmoid LUT to generate the corresponding gating coefficient. For membrane and spike firing, the same PE array is reused to compute the membrane-state update. The

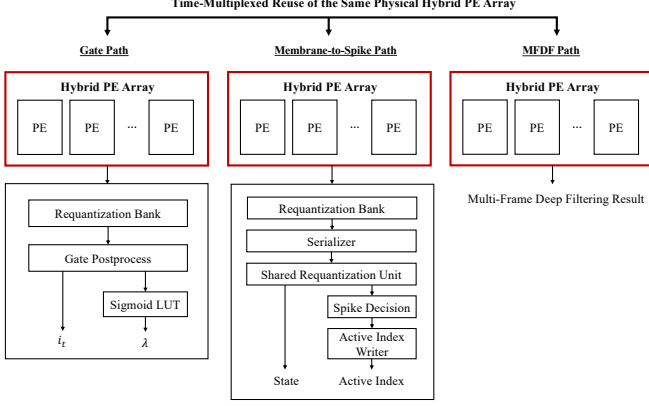


Fig. 3. Time-multiplexed reuse of the same physical Hybrid PE array across different computation paths.

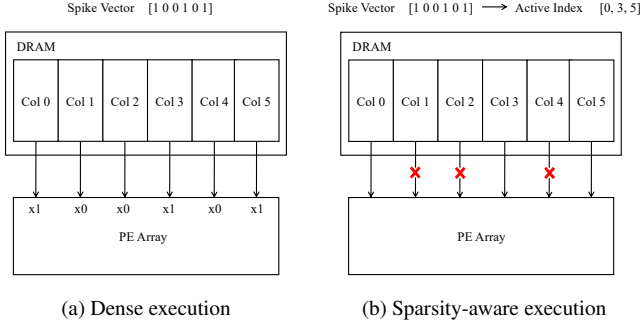


Fig. 4. Comparison between dense matrix-spike execution and the proposed sparsity-aware execution. In the proposed design, only weight vectors corresponding to active spike indices are fetched and accumulated, eliminating DRAM accesses and PE operations associated with inactive spikes.

results are then serialized and processed by the shared requantization unit before spike generation. The spike decision unit identifies active neurons, and their indices are recorded by the active-index writer for subsequent sparse matrix-spike computation. For MFDF, the Hybrid PE array is reused to perform complex MAC accumulation. Real and imaginary components are mapped onto pairs of PEs and accumulated over multiple filter taps, allowing multi-frame deep filtering to be implemented without introducing a dedicated complex arithmetic module. By time-multiplexing the Hybrid PE array across these heterogeneous computation phases, the proposed architecture avoids duplicating arithmetic resources for operation-specific modules. This reuse improves hardware utilization while keeping the flexibility required by the different computation patterns in this neural network.

4.3. Sparsity-Aware Computation

Our spiking neural network exhibits high activation sparsity. To exploit this property, we introduce a sparsity-aware mech-

Table 2. FPGA implementation summary and comparison with speech enhancement FPGA accelerators.

Zephyr FPGA Implementation Summary			
Metric	Value	Metric	Value
Platform	PYNQ-Z1	Device	XC7Z020-1
Clock frequency	100 MHz	Number of PEs	32
Mean latency	5.382 ms	Request/op. skip ^a	87.85%
Traffic reduction ^b	54.47%		

Comparison with Speech Enhancement FPGA Accelerators						
Work	FPGA	LUT	BRAM [†]	DSP	RTF	SNN
Flamis et al. [13]	XC7Z020	30,074	117.5	194	–	✗
Abdullah et al. [14]	XC7A100T	33,910	35.6	0	–	✗
Ni et al. [15]	XCZU48DR	89,987	903	83	0.579	✗
Zephyr	XC7Z020	44,717	78.5	62	0.727	✓

^a Skip zero-valued operations. ^b Reduction in parameter data read from memory. [†] BRAM usage is normalized to 36-Kb BRAM equivalents.

anism that skips inactive neurons during matrix-spike multiplication. Rather than processing all spike entries, only the indices of active spikes are recorded and scheduled for computation, allowing the PE array to fetch and accumulate weights exclusively for active spike events. As a result, computations and DRAM accesses associated with inactive spikes are eliminated. As illustrated in Fig. 4, dense execution process all spike entries regardless of its activity. Consequently, all weight vectors are fetched and sent to PE array even when the corresponding spike value is 0. In contrast, the proposed sparsity-aware mechanism first extracts the indices of all active spike. Only the weight vectors corresponding to active index are fetched and scheduled for computation. Since inactive spike entries are removed before weight fetching, both unnecessary weight access and PE operations are avoided.

5. EXPERIMENTAL SETUP

The accelerator was implemented on a PYNQ-Z1 FPGA with 32 PEs operating at 100 MHz and evaluated using the Intel N-DNS Challenge dataset. Speech-enhancement quality was evaluated using SI-SDR, while hardware performance was measured using board-loop latency and sustained real-time factor. Table 2 summarizes the implementation results of the proposed accelerator as well as comparison against prior works.

6. CONCLUSION

In this work we demonstrate with our system Zephyr that a neuromorphic hardware-software co-designed audio-denoising system is capable of performing high quality audio denoising whilst achieving low power and processing latency.

7. ACKNOWLEDGMENTS

This work was supported by the National Science and Technology Council, Taiwan, under contract no. NSTC 115-2223-E-007-010 and NSTC 115-2640-E-007-014.

8. REFERENCES

- [1] Xiang Hao, Chenxiang Ma, Qu Yang, Jibin Wu, and Kay Chen Tan, “Toward ultralow-power neuromorphic speech enhancement with spiking-fullsubnet,” 2025, vol. 36, pp. 17350–17364, IEEE.
- [2] Tao Sun and Sander Bohté, “Dpsnn: Spiking neural network for low-latency streaming speech enhancement,” *Neuromorphic Computing and Engineering*, vol. 4, no. 4, pp. 044008, 2024.
- [3] Yu Du, Xu Liu, and Yansong Chua, “Spiking structured state space model for monaural speech enhancement,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 766–770.
- [4] Jean-Marc Valin, “A hybrid DSP/deep learning approach to real-time full-band speech enhancement,” *Proc. IEEE MMSP*, 2018.
- [5] Yanxin Hu, Yun Liu, Shubo Lv, Mengtao Xing, Shimin Zhang, Yihui Fu, Jian Wu, Bihong Zhang, and Lei Xie, “DCCRN: Deep complex convolution recurrent network for phase-aware speech enhancement,” in *Proc. Interspeech 2020*, 2020, pp. 2472–2476.
- [6] Xiang Hao, Xiangdong Su, Radu Horaud, and Xiaofei Li, “FullSubNet: A full-band and sub-band fusion model for real-time single-channel speech enhancement,” in *ICASSP 2021 - IEEE International Conference on Acoustics, Speech and Signal Processing*, 2021, pp. 6633–6637.
- [7] Jonathan Timcheck, Sumit Bam Shrestha, Daniel Ben Dayan Rubin, Adam Kupryjanow, Garrick Orchard, Lukasz Pindor, Timothy Shea, and Mike Davies, “The intel neuromorphic dns challenge,” *Neuromorphic Computing and Engineering*, vol. 3, no. 3, pp. 034005, aug 2023.
- [8] H. Dubey, V. Gopal, R. Cutler, A. Aazami, S. Matushevych, S. Braun, S. E. Eskimez, M. Thakker, T. Yoshioka, H. Gamper, and R. Aichner, “ICASSP 2022 deep noise suppression challenge,” in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 9271–9275.
- [9] Xiang Hao and Xiaofei Li, “Fast fullsubnet: Accelerate full-band and sub-band fusion model for single-channel speech enhancement,” *arXiv preprint arXiv:2212.09019*, 2022.
- [10] S. Abdulatif, R. Cao, and B. Yang, “CMGAN: Conformer-based metric-GAN for monaural speech enhancement,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2477–2493, 2024.
- [11] Alessandro Pierro, Steven Abreu, Jonathan Timcheck, Philipp Stratmann, Andreas Wild, and Sumit Bam Shrestha, “Accelerating linear recurrent neural networks for the edge with unstructured sparsity,” in *Proceedings of the 42nd International Conference on Machine Learning*. 2025, vol. 267, pp. 49382–49398, PMLR.
- [12] M. Horowitz, “Computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference (ISSCC)*, 2014, pp. 10–14.
- [13] Georgios Flamis, Stavros Kalapothas, and Paris Kitsoos, “Fpga-soc deployment of complex deep neural network for magnitude and phase computations in denoising of speech signal,” in *2022 IFIP/IEEE 30th International Conference on Very Large Scale Integration (VLSI-SoC)*, 2022, pp. 1–5.
- [14] Salinna Abdullah, Majid Zamani, and Andreas Demosthenous, “Hardware efficient speech enhancement with noise aware multi-target deep learning,” *IEEE Open Journal of Circuits and Systems*, vol. 5, 2024.
- [15] Xianpo Ni, Dongkun Xu, Usman Anwar, and Tughrul Arslan, “A real-time fpga-accelerated causal spectro-temporal reconstruction network for speech enhancement in hearing aids,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2026.